

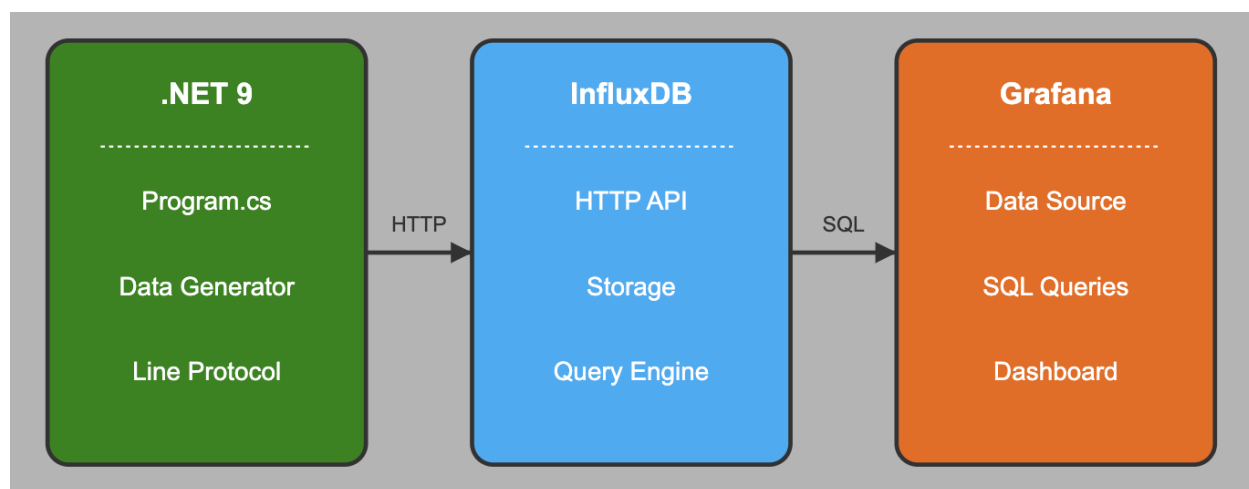
Back to the Future of Time Series Data: Complete Demo Guide



Introduction

This step-by-step guide will help you create the "Back to the Future" themed time series data demo from scratch. This demo showcases how modern time series databases make working with time-stamped data simple and scalable using InfluxDB, .NET, and Grafana.

You'll create a .NET application that generates simulated DeLorean metrics, send them to InfluxDB using Line Protocol format, and visualize them through a Grafana dashboard.



This architecture consists of three main components:

1. **.NET 9 Data Generator** - Simulates metrics from DeLorean time machines
2. **InfluxDB 3 Cloud Serverless** - Stores and processes time series data
3. **Grafana Dashboard** - Provides real-time visualization

Data flows from left to right, starting with the .NET application generating simulated metrics, sending them to InfluxDB using Line Protocol format, and finally being visualized in Grafana using SQL queries.

Prerequisites

Software Requirements

- [.NET 9 SDK](#) - Latest version
- [Visual Studio Code](#) with C# extension installed
- [InfluxDB Cloud Account](#) (Free tier is sufficient)
- [Grafana Cloud Account](#) (Free tier is sufficient)

System Requirements

- **Operating System:** Windows 10/11, macOS, or Linux
- **Memory:** Minimum 2GB RAM (4GB recommended)
- **Storage:** At least 100MB free space
- **Internet Connection:** Required for cloud services

Part 1: InfluxDB Cloud Setup

1. Sign up for InfluxDB Cloud

- Go to [InfluxDB Cloud](#)
- Create a free account or sign in

2. Create a Bucket

- Navigate to "Load Data" > "Buckets"
- Click "Create Bucket"
- Name it "gigawatt_metrics"
- Set retention period to 7 days

3. Generate API Token

- Go to "Load Data" > "API Tokens"
- Click "Generate API Token"
- Select "All Access Token"
- Copy and save this token securely

4. Note Connection Details

- Save your InfluxDB URL (e.g., "https://us-east-1-1.aws.cloud2.influxdata.com")
- Save your organization ID (visible in the URL or in organization settings)
- Save your bucket name ("gigawatt_metrics")

Part 2: Creating the .NET Application from Scratch

Step 1: Create a New .NET Console Project

Open a terminal or command prompt and create a new directory for your project

```
mkdir DeLoreanMonitoring  
cd DeLoreanMonitoring
```

Create a new .NET console application

```
dotnet new console --framework net9.0
```

Add required package references

```
dotnet add package Microsoft.Extensions.Hosting  
dotnet add package Microsoft.Extensions.Http
```

Open the project in Visual Studio Code

```
code .
```

Step 2: Create the Project Structure

Examine and update the generated .csproj file

Update the generated .csproj file to match the following:

```
<Project Sdk="Microsoft.NET.Sdk">  
  
  <PropertyGroup>  
    <OutputType>Exe</OutputType>  
    <TargetFramework>net9.0</TargetFramework>  
    <ImplicitUsings>enable</ImplicitUsings>  
    <Nullable>enable</Nullable>  
  </PropertyGroup>
```

```

    <ItemGroup>
      <PackageReference Include="Microsoft.Extensions.Hosting"
Version="9.0.4" />
      <PackageReference Include="Microsoft.Extensions.Http" Version="9.0.4"
/>
    </ItemGroup>

  </Project>

```

Create the Grafana dashboard directory

```
mkdir -p grafana
```

Step 3: Implementing the Program.cs File

Replace the content of Program.cs with the following

```

using System.Text;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;

namespace DeLoreanMonitoring
{
    public class Program
    {
        public static async Task Main(string[] args)
        {
            var host = Host.CreateDefaultBuilder(args)
                .ConfigureServices((hostContext, services) =>
                {

services.AddHostedService<DeLoreanDataGeneratorService>();
                services.AddHttpClient();
                })
                .Build();

            await host.RunAsync();
        }
    }
}

```

```

    }

    public class DeLoreanDataGeneratorService : BackgroundService
    {
        private readonly HttpClient _httpClient;
        private readonly ILogger<DeLoreanDataGeneratorService> _logger;
        private readonly string _influxUrl;
        private readonly string _influxToken;
        private readonly string _influxOrg;
        private readonly string _influxBucket;

        // Simulated data parameters
        private double _currentVelocity = 0;
        private double _powerLevel = 0;
        private double _temperature = 70;
        private double _plutoniumLevel = 85; // New parameter for Mr.
Fusion
        private double _circuitVoltage = 12.0; // New parameter for Time
Circuits
        private bool _circuitsActive = true; // Track if time circuits
are active
        private string _currentLocation = "Hill Valley";
        private DateTime _targetDate = new DateTime(1985, 10, 26);
        private readonly Random _random = new Random();

        // For cardinality demo
        private readonly string[] _deloreanIds = { "DMC-001", "DMC-002",
"DMC-003", "DMC-004", "DMC-005" };
        private readonly string[] _locations = { "Hill Valley", "Twin Pines
Mall", "Courthouse Square", "Lyon Estates", "Hilldale" };
        private readonly string[] _timePeriods = { "1955", "1985", "2015",
"1885" };
        private readonly string[] _drivers = { "Doc Brown", "Marty McFly",
"Jennifer Parker", "Biff Tannen", "George McFly" };

        public DeLoreanDataGeneratorService(
            IHttpConnectionFactory httpClientFactory,
            ILogger<DeLoreanDataGeneratorService> logger,
            IConfiguration configuration)
        {
            _httpClient = httpClientFactory.CreateClient();
            _logger = logger;

```

```

        // Replace with your actual values
        _influxUrl =
"https://YOUR_INFLUXDB_INSTANCE.aws.cloud2.influxdata.com/api/v2/write";
        _influxToken = "YOUR_INFLUX_TOKEN";
        _influxOrg = "YOUR_ORG_ID";
        _influxBucket = "gigawatt_metrics";
    }

    protected override async Task ExecuteAsync(CancellationToken
stoppingToken)
    {
        _logger.LogInformation("DeLorean Data Generator Service
starting at: {time}", DateTimeOffset.Now);

        while (!stoppingToken.IsCancellationRequested)
        {
            try
            {
                await GenerateAndSendData();
                await Task.Delay(5000, stoppingToken); // Generate data
every 5 seconds
            }
            catch (Exception ex)
            {
                _logger.LogError(ex, "Error generating or sending
data");
            }
        }
    }

    private async Task GenerateAndSendData()
    {
        var timestamp = DateTimeOffset.UtcNow;
        var lines = new List<string>();

        // Generate basic metrics for demo
        UpdateSimulatedValues();

        // Basic DeLorean metrics (low cardinality)
        lines.Add($"delorean_stats,device=flux_capacitor
power_level={_powerLevel} {timestamp.ToUnixTimeMilliseconds() * 1000000}");
        lines.Add($"delorean_stats,device=speedometer
velocity_mph={_currentVelocity} {timestamp.ToUnixTimeMilliseconds() *

```

```

1000000}");
        lines.Add($"delorean_stats,device=engine
temperature_f={_temperature} {timestamp.ToUnixTimeMilliseconds() *
1000000}");
        lines.Add($"delorean_stats,device=navigation
current_location=\"{_currentLocation}\" {timestamp.ToUnixTimeMilliseconds()
* 1000000}");

        // New metrics for the enhanced dashboard
        lines.Add($"delorean_stats,device=mr_fusion
plutonium_level={_plutoniumLevel} {timestamp.ToUnixTimeMilliseconds() *
1000000}");
        lines.Add($"delorean_stats,device=time_circuits
voltage={_circuitVoltage} {timestamp.ToUnixTimeMilliseconds() * 1000000}");
        lines.Add($"delorean_stats,device=time_circuits
status=\"({_circuitsActive ? \"active\" : \"inactive\"})\"
{timestamp.ToUnixTimeMilliseconds() * 1000000}");

        // High cardinality data - generate many unique tag
combinations
        for (int i = 0; i < 20; i++)
        {
            string deloreanId =
                _deloreanIds[_random.Next(_deloreanIds.Length)];
            string location =
                _locations[_random.Next(_locations.Length)];
            string timePeriod =
                _timePeriods[_random.Next(_timePeriods.Length)];
            string driver = _drivers[_random.Next(_drivers.Length)];
            double fuelLevel = Math.Round(_random.NextDouble() * 100,
2);

            double temporalStability = Math.Round(75 +
                _random.NextDouble() * 25, 2);

            // This creates high cardinality with many tag combinations

            lines.Add($"delorean_detailed,id={deloreanId},location={location},time_peri
od={timePeriod},driver={driver} " +

            $"fuel_level={fuelLevel},temporal_stability={temporalStability},velocity={M
ath.Round(_random.NextDouble() * 90, 1)} " +
                $"{timestamp.AddMilliseconds(i *
100).ToUnixTimeMilliseconds() * 1000000}");

```

```

    }

    // Join all lines and send to InfluxDB
    string payload = string.Join("\n", lines);
    await SendToInfluxDB(payload);

    _logger.LogInformation($"Generated {lines.Count} data points at
{DateTimeOffset.Now}");
}

private void UpdateSimulatedValues()
{
    // Simulate time travel event if velocity approaching 88 mph
    if (_currentVelocity > 85)
    {
        _currentVelocity = _random.NextDouble() * 30; // Slow down
after time travel
        _powerLevel = _random.NextDouble() * 20;      // Power
drain after time travel
        _plutoniumLevel = Math.Max(0, _plutoniumLevel -
(_random.NextDouble() * 15)); // Consume plutonium during time travel

        // Simulate time circuit instability during time travel
        _circuitVoltage = 8.0 + (_random.NextDouble() * 3.0);

        // Small chance time circuits might fail after time travel
        _circuitsActive = _random.NextDouble() > 0.1;

        _currentLocation =
_locations[_random.Next(locations.Length)];
        _targetDate = _targetDate.AddYears(_random.Next(-100,
100));

        _logger.LogInformation("⚡ TIME TRAVEL EVENT DETECTED!
⚡");
    }
    else
    {
        // Otherwise make small adjustments to current values
        _currentVelocity = Math.Min(88, _currentVelocity +
(_random.NextDouble() * 10 - 3));
        _powerLevel = Math.Min(100, _powerLevel +
(_random.NextDouble() * 10 - 3));
    }
}

```



```

        // Gradually decrease plutonium level with normal operation
        _plutoniumLevel = Math.Max(0, _plutoniumLevel -
(_random.NextDouble() * 0.5));

        // If plutonium gets too low, simulate a refueling
        if (_plutoniumLevel < 10 && _random.NextDouble() > 0.7)
        {
            _plutoniumLevel = 85 + (_random.NextDouble() * 15);
            _logger.LogInformation("Mr. Fusion Refueled!");
        }

        // Time circuits voltage fluctuations
        if (_circuitsActive)
        {
            // Normal voltage is around 12-13V with minor
fluctuations
            _circuitVoltage = 12.0 + (_random.NextDouble() * 1.0 -
0.5);

            // Randomly simulate a voltage spike or drop
            if (_random.NextDouble() > 0.9)
            {
                if (_random.NextDouble() > 0.5)
                {
                    // Voltage spike
                    _circuitVoltage = 13.5 + (_random.NextDouble()
* 2.0);
                }
                else
                {
                    // Voltage drop
                    _circuitVoltage = 10.0 - (_random.NextDouble()
* 1.5);
                }
            }
        }
        else
        {
            // Circuits inactive - low voltage with minor noise
            _circuitVoltage = 1.0 + (_random.NextDouble() * 0.5);

            // Chance to reactivate circuits

```

```

        if (_random.NextDouble() > 0.8)
        {
            _circuitsActive = true;
            _logger.LogInformation("Time Circuits Activated!");
        }
    }

    // Occasionally spike power when approaching 88 mph
    if (_currentVelocity > 80)
    {
        _powerLevel = 121.0; // 1.21 gigawatts!
    }

    _temperature = Math.Max(60, Math.Min(250, _temperature +
(_random.NextDouble() * 8 - 4)));
    }
}

private async Task SendToInfluxDB(string lineProtocolData)
{
    try
    {
        HttpRequestMessage request = new
HttpRequestMessage(HttpMethod.Post,

$"{_influxUrl}?org={_influxOrg}&bucket={_influxBucket}&precision=ns");

        request.Headers.Add("Authorization", $"Token
{_influxToken}");
        request.Content = new StringContent(lineProtocolData,
Encoding.UTF8, "text/plain");

        var response = await _httpClient.SendAsync(request);

        if (!response.IsSuccessStatusCode)
        {
            string errorContent = await
response.Content.ReadAsStringAsync();
            _logger.LogError($"Failed to send data to InfluxDB:
{response.StatusCode}, {errorContent}");
        }
    }
    catch (Exception ex)

```

```

        {
            _logger.LogError(ex, "Exception while sending data to
InfluxDB");
        }
    }
}

```

Update the InfluxDB connection details

1. Replace `YOUR_INFLUXDB_INSTANCE` with your InfluxDB Cloud URL
2. Replace `YOUR_INFLUX_TOKEN` with your API token
3. Replace `YOUR_ORG_ID` with your organization ID

Save the file

Step 4: Build and Run the Application

Build the application

```
dotnet build
```

Run the application

```
dotnet run
```

Verify operation

- You should see console output showing data points being generated every 5 seconds
 - Look for log messages like "Generated X data points at [timestamp]"
 - Occasionally, you'll see special events like "⚡ TIME TRAVEL EVENT DETECTED! ⚡" or "Mr. Fusion Refueled!"
2. **Let the application run**
 - Allow it to generate data for at least 5 minutes to populate enough historical data for the dashboard

Step 5: Verify Data in InfluxDB

1. **Log in to your InfluxDB Cloud account**
2. **Navigate to "Data Explorer"**
3. **Select your "gigawatt_metrics" bucket**

Run this query to verify data is being stored

```
SELECT *FROM "delorean_stats"WHERE time >= now() - interval '15 minutes'LIMIT 100
```

Examine the results

- You should see metrics for flux_capacitor, speedometer, engine, and other devices
- Check that each device has the expected fields (power_level, velocity_mph, temperature_f, etc.)

Part 3: Creating the Grafana Dashboard

Step 1: Create the Grafana Data Source

Sign in to your Grafana Cloud account

Navigate to Configuration > Data Sources

Click "Add data source" and select "InfluxDB"

Configure with these settings:

- **Name:** DeLorean_Monitoring
- **URL:** Your InfluxDB Cloud URL (without the /api/v2/write path)
- **Query Language:** SQL
- **Database:** gigawatt_metrics
- **Token:** Your InfluxDB API token

Add these HTTP Headers:

- Name: Authorization, Value: Token YOUR_INFLUX_TOKEN
- Name: Content-Type, Value: application/json
- Name: Accept, Value: application/json

Click "Save & Test"

- You should see "Data source is working" in green

Step 2: Create the Dashboard Manually

Create a new dashboard

- Click on "Dashboards" in the left sidebar
- Click "Create Dashboard"
- Click "Add Visualization"

Set dashboard properties

- Click the gear icon in the upper right
- Dashboard title: "DeLorean DMC-12 Time Machine Monitoring"
- Set refresh rate: 5s
- Time range: Last 15 minutes
- Click "Save"

Step 3: Create the "Flux Capacitor Power" Panel

Add a new panel

- Click "Add panel" > "Add new panel"
- Select your "DeLorean_Monitoring" data source

Configure the query

- Switch to "Code" editor mode
- Enter this SQL query:

```
SELECT time, power_level
FROM "delorean_stats"
WHERE device = 'flux_capacitor' AND time >= now() - interval '15 minutes'
```

Configure visualization

- Select "Gauge" from the visualization dropdown
- Title: "Flux Capacitor Power (1.21 Gigawatts = 121)"
- In the Field tab:
 - Unit: watt
 - Min: 0
 - Max: 150
 - Set thresholds:
 - 0-50: green
 - 50-100: yellow
 - 100-121: orange
 - 121+: red/purple

Save the panel

- Click "Apply"

Step 4: Create the "Current Speed / Time Travel Threshold" Panel

Add a new panel

- Click "Add panel" > "Add new panel"
- Select your "DeLorean_Monitoring" data source

Configure the query

```
SELECT time, velocity_mph
FROM "delorean_stats"
WHERE device = 'speedometer' AND time >= now() - interval '5 minutes'
```

Configure visualization

- Select "Stat" from the visualization dropdown
- Title: "Current Speed / Time Travel Threshold"
- In the Field tab:
 - Unit: velocitymph (or create custom unit)
 - Set thresholds:
 - 0-60: green
 - 60-75: yellow
 - 75-85: orange
 - 85+: red
- In the Display tab:
 - Graph mode: Area
- Show: Calculate last value

Save the panel

- Click "Apply"

Step 5: Create the "Mr. Fusion Fuel Level" Panel

Add a new panel

- Click "Add panel" > "Add new panel"
- Select your "DeLorean_Monitoring" data source

Configure the query

```
SELECT time, plutonium_level
```

```
FROM "delorean_stats"  
WHERE device = 'mr_fusion' AND time >= now() - interval '15 minutes'
```

Configure visualization

- Select "Gauge" from the visualization dropdown
- Title: "Mr. Fusion Fuel Level"
- In the Field tab:
 - Unit: percent
 - Min: 0
 - Max: 100
 - Set thresholds:
 - 0-25: red/orange
 - 25-50: yellow
 - 50-75: light blue
 - 75+: purple

Save the panel

- Click "Apply"

Step 6: Create the "Time Circuits Power Stability" Panel

Add a new panel

- Click "Add panel" > "Add new panel"
- Select your "DeLorean_Monitoring" data source

Configure the query

```
SELECT time, voltage  
FROM "delorean_stats"  
WHERE device = 'time_circuits' AND time >= now() - interval '15 minutes'
```

Configure visualization

- Select "Time series" from the visualization dropdown
- Title: "Time Circuits Power Stability"
- In the Field tab:
 - Unit: volt
 - Set appropriate thresholds
- In the Graph styles tab:
 - Fill opacity: 20%
 - Line width: 2

- Line interpolation: Linear

Save the panel

- Click "Apply"

Step 7: Create the "Engine Temperature" Panel

Add a new panel

- Click "Add panel" > "Add new panel"
- Select your "DeLorean_Monitoring" data source

Configure the query

```
SELECT time, temperature_f
FROM "delorean_stats"
WHERE device = 'engine' AND time >= now() - interval '15 minutes'
```

Configure visualization

- Select "Stat" from the visualization dropdown
- Title: "Engine Temperature"
- In the Field tab:
 - Unit: fahrenheit
 - Set thresholds:
 - 0-150: green
 - 150-200: orange
 - 200+: red
- In the Display tab:
 - Graph mode: Area
 - Show: Calculate last value

Save the panel

- Click "Apply"

Step 8: Create the "DeLorean Velocity" Panel

Add a new panel

- Click "Add panel" > "Add new panel"
- Select your "DeLorean_Monitoring" data source

Configure the query

```
SELECT time, velocity_mph
FROM "delorean_stats"
WHERE device = 'speedometer' AND time >= now() - interval '15 minutes'
```

Configure visualization

- Select "Time series" from the visualization dropdown
- Title: "DeLorean Velocity (88 mph Time Travel Threshold)"
- In the Field tab:
 - Unit: velocitymph (or custom unit)
 - Set threshold at 88 with color gold/yellow
- In the Graph styles tab:
 - Fill opacity: 20%
 - Line width: 3
 - Line interpolation: Smooth
 - Gradient mode: Scheme
- In the Thresholds tab:
 - Add threshold at 88 mph
 - Style: Line + Area

Save the panel

- Click "Apply"

Step 9: Arrange the Dashboard Layout

Resize and arrange panels

- Click and drag panel edges to resize
- Click and drag panel headers to move
- Create a 2x3 grid layout:
 - Top row: Flux Capacitor Power, Current Speed, Mr. Fusion Fuel Level
 - Bottom row: Time Circuits Stability, Engine Temperature, DeLorean Velocity

Add a header (optional)

- Click "Add panel" > "Add a new row"
- Title it "DeLorean Time Machine Metrics"
- Expand/collapse as needed

Set dashboard time range

- Top right: Select "Last 15 minutes"

- Enable auto-refresh every 5s

Save the final dashboard

- Click the save icon (or CTRL+S)
- Add a description: "Real-time monitoring of DeLorean time machine metrics"
- Add tags: delorean, time-series, back-to-the-future
- Click "Save"

Part 4: Key Demo Talking Points

Now that your dashboard is complete and data is flowing, you're ready to demonstrate the three key takeaways from your presentation:

1. Handling High Cardinality

Explain using the dashboard:

- Point out how the application generates data for multiple DeLoreans (DMC-001 through DMC-005)
- Each DeLorean has multiple tag combinations (location, time period, driver)
- Show how InfluxDB efficiently handles these many unique tag combinations
- Explain how tagging strategy impacts query performance (device as a tag enables filtering)

2. Efficient Writing with Line Protocol and Reading with SQL

Explain using the code and dashboard:

- Show the Line Protocol format in the code and how compact it is
- Point out the batch writing approach that groups multiple data points in a single request
- Show the SQL queries powering each dashboard panel
- Demonstrate how time filtering (`time >= now() - interval 'x minutes'`) improves performance

3. Scaling from Thousands to Millions of Data Points

Explain using the architecture:

- Show how each component (DeLorean metrics generator, InfluxDB, Grafana) plays a role in scalability
- Explain how time-bucketing aggregation could be used for long-term analysis
- Mention techniques like downsampling older data and multi-tier storage
- Point out how the dashboard maintains performance even as data accumulates

Panel Talking Points

When presenting the dashboard, use these talking points to connect each panel to time series concepts:

Flux Capacitor Power Panel

- Shows real-time monitoring of current status
- Demonstrates instant visibility into system state
- Critical threshold at 121 watts (1.21 gigawatts)
- Represents the "present" view of time series data

Current Speed Panel

- Shows proximity to the time travel threshold
- Demonstrates how thresholds can indicate critical states
- Area graph shows recent trend context
- Represents the "immediate future" perspective

Mr. Fusion Fuel Level Panel

- Shows resource availability monitoring
- Demonstrates percentage-based gauges for capacity tracking
- Represents forecasting and resource planning aspects

Time Circuits Voltage Panel

- Shows time series data over time
- Demonstrates pattern identification for normal vs. abnormal behavior
- Voltage stability critical for time travel accuracy
- Represents the "temporal context" of the system

Engine Temperature Panel

- Shows current value with historical context
- Demonstrates how recent trends inform current understanding
- Temperature trend indicates potential future issues
- Represents the "present with context" perspective

DeLorean Velocity Chart

- Shows historical patterns over 15 minutes
- Demonstrates event detection (88 mph threshold crossings)
- Each spike and drop represents a time travel event

- Represents the "historical analysis" perspective

Troubleshooting Tips

If you encounter issues during the demo creation or presentation:

Application Not Generating Data

- Check InfluxDB URL, token, and organization ID
- Verify network connectivity
- Check console output for error messages

No Data in Grafana Panels

- Verify data exists in InfluxDB using Data Explorer
- Check query syntax (case-sensitive field and device names)
- Verify time range includes data points

Panel Visualization Issues

- Adjust min/max ranges for gauges
- Check unit settings
- Verify threshold configurations
- Try refreshing the dashboard

Project Structure

- **Program.cs**: Main .NET console application for generating time series data
- **DeLoreanMonitoring.csproj**: Project file for the .NET application
- **grafana/delorean_monitoring_dashboard.json**: Grafana dashboard configuration
- **architecture-diagram.png**: Visual representation of the system architecture
- **bin/**: Build output directory
- **obj/**: Intermediate build files

Additional Resources

- [InfluxDB Cloud Documentation](#)
- [InfluxDB Line Protocol Reference](#)
- [Grafana Dashboard Documentation](#)
- [.NET 9 Documentation](#)

Final Thoughts

This demo effectively illustrates the power of time series data for real-time monitoring, historical analysis, and future prediction - all through the fun lens of the Back to the Future theme.

By building this from scratch, you've gained hands-on experience with the complete workflow of time series data:

1. Generation and collection of metrics
2. Efficient storage using specialized time series databases
3. Analysis and visualization using SQL queries
4. Real-time dashboarding of complex time-based patterns

The same principles demonstrated here can be applied to real-world applications like IoT monitoring, application performance tracking, financial analysis, and much more.

Happy time traveling through your data!